

Reviewer Pack — Minimal Rank of Deligne-Lusztig Classes in Representation Th...

Entry 5b4e726ce8cd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 21:18:30 UTC

Minimal Rank of Deligne-Lusztig Classes in Representation Theory over Resolution Proof Trees

Entry ID: 5b4e726ce8cd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 21:18:30 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Deligne-Lusztig Constructions) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For a given k -CNF formula F with n variables, the minimal rank of the Deligne-Lusztig class associated with the vector space of its resolution proofs is $\Theta(n^{1/2 + \varepsilon})$ for some constant $\varepsilon > 0$.’, ‘This statement implies that the complexity of finding resolution proofs for F grows exponentially in the size of F .’, ‘For a counterexample, find a k -CNF formula with a Deligne-Lusztig class rank less than $n^{1/4}$.’]

Rationale (proposer’s reasoning):

[‘The Deligne-Lusztig theory provides a rich structure for studying representations of groups, and its application to complexity theory could potentially reveal new insights into the nature of resolution proof complexity.’, ‘If the conjecture holds, it would suggest that the resolution proofs for complex formulas have an intricate algebraic structure, which could be exploited in proving lower bounds on their size.’, ‘The minimal rank as a measure of complexity is closely related to the algebraic geometry of moduli spaces, offering a new angle for understanding proof complexity.’]

Taxonomy category: DELIGNE_LUSZTIG_CLASS_RANK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 927c13ac2c1b166f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of Deligne-Lusztig classes in the vector space of resolution proofs for k -CNF formulas with n variables is measured as $\Theta(n^{1/2 + \varepsilon})$ where $\varepsilon > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Deligne-Lusztig classes' AND 'resolution proof complexity' - 'Representation theory' AND 'minimal rank' AND 'proof trees' - 'CNF formula' AND 'Deligne-Lusztig class' AND 'exponential growth'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
            continue
        for j in range(n):
            A[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
```

```

return C

def deligne_lusztig_rank(k, n):
    if k != 2:
        return "mapping_undefined"

    # Generate a random k-CNF formula with n variables
    clauses = []
    for _ in range(10): # Each clause has at most 3 literals
        clause = set()
        while len(clause) < 3:
            var = random.randint(1, n)
            if random.choice([True, False]):
                clause.add(var)
            else:
                clause.add(-var)
        clauses.append(list(clause))

    # Construct the polynomial representing the formula
    poly = [[0] * (n + 1) for _ in range(n + 1)]
    poly[0][0] = 1
    for clause in clauses:
        term = [1]
        for lit in clause:
            if lit > 0:
                term = [p + t * x[lit-1] for p, t, x in zip(poly[-1], term, x)]
            else:
                term = [p - t * x[-lit-1] for p, t, x in zip(poly[-1], term, x)]
        poly.append(term)

    # Compute the Deligne-Lusztig rank
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(i, n + 1):
            if i == j:
                A[i][j] = sum(poly[k][i-1] * poly[k][j-1] for k in range(n))
            else:
                A[i][j] = sum(poly[k][i-1] * poly[k][j-1] for k in range(n)) - sum(poly[k][i-2] *

gaussian_elimination(A)
rank = 0
for row in A:
    if any(row):
        rank += 1

return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    ranks = []

    for n in n_values:
        rank = deligne_lusztig_rank(2, n)
        if rank == "mapping_undefined":
            return {

```

```

        "metric_name": "deligne_lusztig_rank",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
    ranks.append(rank)

mean_rank = sum(ranks) / len(ranks)
std_rank = math.sqrt(sum((x - mean_rank) ** 2 for x in ranks) / len(ranks))
conjecture_holds = all(rank >= n**0.5 for rank, n in zip(ranks, n_values))

return {
    "metric_name": "deligne_lusztig_rank",
    "metric_value": mean_rank,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"rank<{n**0.5} for n={n}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30 * 2 + 1, 2))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank<{n**0.5} for n={n}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d503d9b.py", line 126, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d503d9b.py", line 97, in run_trial

```

    rank = deligne_lusztig_rank(2, n)

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8d503d9b.py", line 71, in deligne_luszt
    term = [p - t * x-lit-1] for p, t, x in zip(poly[-1], term, x)]
```

NameError: name 'x' is not defined

8. Critic adversarial review